

Trilha de Node.js

Módulo 01: Introdução

Instruções para a melhor prática de Estudo

1. **Leia atentamente todo o conteúdo:** Antes de iniciar qualquer atividade, faça uma leitura detalhada do material fornecido na trilha, compreendendo os conceitos e os exemplos apresentados.
2. **Não se limite ao material da trilha:** Utilize o material da trilha como base, mas busque outros materiais de apoio, como livros, artigos acadêmicos, vídeos, e blogs especializados. Isso enriquecerá o entendimento sobre o tema.
3. **Explore a literatura:** Consulte livros e publicações reconhecidas na área, buscando expandir seu conhecimento além do que foi apresentado. A literatura acadêmica oferece uma base sólida para a compreensão de temas complexos.
4. **Realize todas as atividades propostas:** Conclua cada uma das atividades práticas e teóricas, garantindo que você esteja aplicando o conhecimento adquirido de maneira ativa.
5. **Evite o uso de Inteligência Artificial para resolução de atividades:** Utilize suas próprias habilidades e conhecimentos para resolver os exercícios. O aprendizado vem do esforço e da prática.
6. **Participe de debates:** Discuta os conteúdos estudados com professores, colegas e profissionais da área. O debate enriquece o entendimento e permite a troca de diferentes pontos de vista.
7. **Pratique regularmente:** Não deixe as atividades para a última hora. Pratique diariamente e revise o conteúdo com frequência para consolidar o aprendizado.
8. **Peça feedback:** Solicite o retorno dos professores sobre suas atividades e participe de discussões sobre os erros e acertos, utilizando o feedback para aprimorar suas habilidades.

Essas instruções são fundamentais para garantir um aprendizado profundo e eficaz ao longo das trilhas.

Módulo 1: Introdução ao Node.js

1.1 O que é Node.js?

Node.js é uma plataforma de execução de JavaScript baseada no motor V8 do Google, projetada para criar aplicações de alto desempenho e escaláveis. Com o Node.js, é possível executar códigos JavaScript fora do navegador, o que o torna ideal para aplicações do lado do servidor.

Diferente do JavaScript executado no navegador, o Node.js permite acesso ao sistema de arquivos, rede, processos e variáveis de ambiente, sendo ideal para aplicações backend.

Principais Características

- **Arquitetura Single-threaded:** Processa várias requisições simultaneamente usando o modelo non-blocking.
- **Programação Assíncrona:** Utiliza callbacks, promises e async/await para lidar com operações de I/O.
- **Eficiente:** Ideal para aplicações em tempo real, como chats e streaming.
- **Grande Ecossistema:** A plataforma utiliza o npm (Node Package Manager), que possui milhares de bibliotecas.
- **Motor V8:** Responsável por converter código JavaScript em código máquina de forma eficiente.

Importância e Aplicações

- Desenvolvimento de APIs REST e GraphQL.
- Servidores web e aplicações em tempo real (WebSockets).
- Sistemas de fila e processamento em segundo plano.
- Microserviços escaláveis e flexíveis.

Exemplo de Uso

```
console.log('Node.js está funcionando!');
```

Execute o comando:

```
node arquivo.js
```

1.2 Arquitetura e Modelo de Execução do Node.js

1.2.1 Visão Geral: Como o Node.js Funciona

O Node.js utiliza um modelo de execução **assíncrono e orientado a eventos**, baseado no **Event Loop**.

Esse modelo permite que a aplicação processe múltiplas requisições simultaneamente sem bloquear o fluxo principal de execução, mesmo utilizando apenas um thread principal.

Em vez de criar uma thread para cada requisição (modelo tradicional), o Node.js delega operações demoradas (como acesso a arquivos, rede ou timers) para mecanismos internos, liberando o thread principal para continuar processando outras tarefas.

1.2.2 Componentes Fundamentais da Arquitetura

Event Loop

É o coração do Node.js. Ele monitora continuamente as filas de eventos e decide quando cada callback pode ser executado no thread principal.

Call Stack (Pilha de Execução)

Responsável pela execução do código síncrono. As funções são empilhadas e executadas em ordem até que a pilha esteja vazia.

Thread Pool

Conjunto de threads auxiliares utilizadas para executar tarefas de I/O mais pesadas, como:

- Leitura e escrita em arquivos
- Operações criptográficas
Compressão
- Algumas operações de rede

Callback Queue (Fila de Callbacks)

Fila onde os callbacks de operações assíncronas aguardam para serem processados quando a Call Stack estiver livre.

Motor V8

Motor JavaScript desenvolvido pelo Google, responsável por interpretar e converter o código JavaScript em código de máquina de forma altamente eficiente.

1.2.3 Conceitos Essenciais

Evento

Qualquer ação detectada pelo sistema, como:

- Uma requisição HTTP
- Um timer (`setTimeout`)
- Leitura de um arquivo
- Conexão de um cliente

Callback

Função que será executada automaticamente quando uma operação assíncrona for concluída.

Thread

Unidade básica de execução.

No Node.js:

- O código JavaScript roda em **um único thread principal**
- Operações de I/O podem utilizar threads do **Thread Pool**

Queue (Fila)

Estrutura responsável por armazenar tarefas pendentes até que possam ser executadas pelo Event Loop.

1.2.4 Modelo Single-threaded vs Multi-threaded**Single-threaded (Node.js)**

- Um único thread principal executa o código JavaScript
- Ideal para aplicações com muitas operações de I/O
- Menor consumo de recursos
- Evita problemas comuns de concorrência

Multi-threaded (Modelo tradicional)

- Múltiplos threads executam tarefas simultaneamente
- Maior complexidade de sincronização
- Usado em cenários específicos no Node.js, como com o módulo `worker_threads`

1.2.5 Fluxo Simplificado de Execução

1. Uma requisição ou evento chega à aplicação
2. O código síncrono é executado na **Call Stack**
3. Operações demoradas são enviadas para o **Thread Pool**
4. Após a conclusão, o callback é colocado na **Callback Queue**
5. O **Event Loop** move o callback para a Call Stack quando ela estiver livre
6. O código do callback é executado

Exemplo de Execução Assíncrona

```
setTimeout(() => {  
  console.log('Essa mensagem aparece depois de 2 segundos');  
}, 2000);  
  
console.log('Essa mensagem aparece primeiro');
```

Saída:

```
Essa mensagem aparece primeiro  
Essa mensagem aparece depois de 2 segundos
```

📌 **Observação importante:** Mesmo o `setTimeout` aparecendo antes no código, sua execução ocorre depois, pois ele é tratado como uma operação assíncrona gerenciada pelo Event Loop.

1.3 Configurando o Ambiente de Desenvolvimento

Instalação do Node.js

1. Acesse o site oficial: [Node.js](https://nodejs.org).
2. Baixe a versão recomendada (LTS).
3. Verifique a instalação:

```
node -v  
npm -v
```

Configuração do Editor de Código

- **Recomendado:** VS Code.
- Instale a extensão "Node.js Extension Pack".

Criando seu Primeiro Projeto

1. Crie uma pasta e inicialize o projeto:

```
mkdir meu-projeto  
cd meu-projeto  
npm init -y
```

2. Adicione um arquivo `index.js` com o seguinte conteúdo:

```
console.log('Meu primeiro projeto Node.js!');
```

3. Execute o projeto:

```
node index.js
```

1.4 Gerenciamento de Pacotes com npm

O que é npm?

O npm (Node Package Manager) é o gerenciador de pacotes oficial do Node.js. Ele permite adicionar bibliotecas externas ao seu projeto, facilitando o desenvolvimento.

Comandos Principais

- Instalar uma biblioteca:

```
npm install nome-da-biblioteca
```

- Instalar como dependência de desenvolvimento:

```
npm install nome-da-biblioteca --save-dev
```

- Listar dependências:

```
npm list
```

- Remover uma biblioteca:

```
npm uninstall nome-da-biblioteca
```

O Arquivo `package.json`

O `package.json` é o arquivo de configuração do seu projeto Node.js, contendo informações como:

- Nome e versão do projeto.
- Dependências e dependências de desenvolvimento.
- Scripts customizados para executar tarefas automatizadas.

Exemplo Prático

1. Instale o pacote `chalk` para colorir saídas no terminal:

```
npm install chalk
```

2. Modifique o `index.js`:

```
const chalk = require('chalk');  
console.log(chalk.green('Texto em verde!'));
```

3. Execute o arquivo:

```
node index.js
```

Explorando o Ecossistema npm

O npm possui um vasto repositório de pacotes. Alguns pacotes populares incluem:

1. **express**: Framework para criação de servidores.
 2. **axios**: Cliente HTTP para realizar requisições a APIs.
 3. **dotenv**: Gerenciamento de variáveis de ambiente.
-

Lista de Exercícios

Questões Teóricas

1. Explique o que é Node.js e suas principais características.
2. Qual é a diferença entre o modelo single-threaded e o multi-threaded?
3. O que é o Event Loop no Node.js?
4. Liste três aplicações onde o Node.js é amplamente utilizado.
5. Como o npm facilita o desenvolvimento de aplicações?
6. Quais os passos para criar um projeto Node.js?
7. Cite dois exemplos de pacotes populares no npm e suas utilidades.
8. O que é o arquivo **package.json** e qual sua importância?
9. Explique o conceito de callback no Node.js.
10. Qual é o papel do motor V8 no Node.js?

Questões Práticas

- Instale o Node.js e verifique a versão instalada.
- Crie um projeto Node.js e exiba a mensagem "Bem-vindo ao Node.js!".
- Escreva um código que exiba uma mensagem com um atraso de 3 segundos.
- Instale o pacote **axios** e use-o para fazer uma requisição GET para uma API.
- Use o pacote **chalk** para exibir uma mensagem de erro em vermelho no terminal.
- Crie um script que leia um arquivo **texto.txt** e exiba o conteúdo no console.
- Escreva uma função que use callbacks para somar dois números.
- Adicione uma dependência de desenvolvimento ao seu projeto e verifique-a no arquivo **package.json**.
- Crie um arquivo JSON com informações de um usuário e leia-o usando Node.js.
- Desenvolva um script que exiba a data e hora atuais formatadas no terminal.